

AI Attendance Kiosk: Technical Code Explainer

Biswajyoti Nath DITEC Presentation Script

July 2026

Contents

1	Introduction to the Presenter Script	2
2	Security: Next.js Edge Middleware	2
2.1	Why Edge Middleware?	2
2.2	Code Breakdown: middleware.js	2
3	Concurrency: Mitigating Race Conditions	3
3.1	Code Breakdown: pages/api/attendance/index.js	3
4	Performance: Web Worker Offloading	4
4.1	Code Breakdown: workers/faceWorker.js	4

1 Introduction to the Presenter Script

This document serves as a deep-dive technical script designed to answer rigorous engineering questions during the DITEC internship presentation. It breaks down the most complex logic implementations: Concurrency Management (Mutex Locks), Web Worker Offloading for Edge AI, and Edge Security (Middleware Auth).

2 Security: Next.js Edge Middleware

To protect the administrative dashboard from unauthorized access, we implemented HTTP Basic Authentication at the Edge layer.

2.1 Why Edge Middleware?

Instead of securing routes inside React components (which can cause flashes of unauthorized content) or at the Node.js API layer (which wastes server cycles), we intercept requests at the Edge. If authentication fails, the request never hits the main server.

2.2 Code Breakdown: middleware.js

```
1 import { NextResponse } from 'next/server';
2
3 export function middleware(req) {
4   const basicAuth = req.headers.get('authorization');
5   const url = req.nextUrl;
6
7   // 1. Intercept Target Routes
8   if (url.pathname.startsWith('/admin') || url.pathname.startsWith('/api/users')) {
9
10    // 2. Validate Credentials
11    if (basicAuth) {
12      const authValue = basicAuth.split(' ')[1];
13      const [user, pwd] = atob(authValue).split(':');
14
15      const expectedUser = process.env.ADMIN_USER;
16      const expectedPass = process.env.ADMIN_PASS;
17
18      if (user === expectedUser && pwd === expectedPass) {
19        return NextResponse.next(); // Access Granted
20      }
21    }
22
23    // 3. Trigger Native Browser Prompt
24    url.pathname = '/api/auth';
25    return new NextResponse('Auth Required.', {
26      status: 401,
27      headers: {
28        'WWW-Authenticate': 'Basic realm="Secure Admin Dashboard"',
29      },
30    });
31  }
32  return NextResponse.next();
```

Explainer Points for Jury:

- **Line 8:** We strictly target `/admin` (Frontend) and `/api/users` (Backend) to ensure complete protection.
- **Line 25-29:** By responding with a 401 and the `WWW-Authenticate` header, we force the browser to trigger its native, un-hackable password popup.
- **Line 15-16:** Credentials are dynamically injected via Docker environment variables, ensuring no hardcoded secrets exist in the codebase.

3 Concurrency: Mitigating Race Conditions

A classic Time-of-Check-to-Time-of-Use (TOCTOU) vulnerability exists if a user holds their face to the camera too long, causing the frontend to fire two API POST requests in rapid succession.

3.1 Code Breakdown: `pages/api/attendance/index.js`

We utilize a backend In-Memory Mutex Lock using a standard JavaScript Map.

```

1 // Global in-memory map to act as a mutex lock
2 const activeLocks = new Map();
3
4 export default async function handler(req, res) {
5   if (req.method === 'POST') {
6     const { userId, type } = req.body;
7
8     // 1. Check and Acquire Lock
9     if (activeLocks.has(userId)) {
10      return res.status(409).json({ error: "Currently processing."
11    });
12    }
13    activeLocks.set(userId, true);
14
15    try {
16      // ... Execute database SELECTs and INSERTs ...
17    } catch (error) {
18      console.error(error);
19    } finally {
20      // 2. Release Lock
21      activeLocks.delete(userId);
22    }
23  }
24 }
```

Explainer Points for Jury:

- **Line 2 (Global Scope):** `activeLocks` persists across API calls because Next.js reuses the Node.js process environment in production.
- **Line 9-10 (Mutex Check):** If a second request for the same user arrives while the first is running, it instantly hits the 409 `Conflict` and is aborted.

- **Line 19-20 (Guaranteed Release):** We place `activeLocks.delete()` inside a `finally` block. Even if the database crashes or the logic throws an error, the lock is strictly released to prevent deadlocks.

4 Performance: Web Worker Offloading

Running facial recognition inference (`face-api.js`) on the main thread causes the React UI to freeze. To achieve 60fps, we isolated the AI models into a background Web Worker.

4.1 Code Breakdown: `workers/faceWorker.js`

```
1 import * as faceapi from 'face-api.js';
2
3 self.onmessage = async (e) => {
4   const { imageBuffer, width, height } = e.data;
5
6   // 1. Reconstruct Image Data from Buffer
7   const imageData = new ImageData(
8     new Uint8ClampedArray(imageBuffer),
9     width, height
10  );
11
12  // 2. Offscreen Inference
13  const detection = await faceapi
14    .detectSingleFace(imageData, new faceapi.TinyFaceDetectorOptions
15    ())
16    .withFaceLandmarks()
17    .withFaceDescriptor();
18
19  // 3. Post Results Back to UI
20  if (detection) {
21    self.postMessage({ descriptor: detection.descriptor });
22  }
23 };
```

Explainer Points for Jury:

- **Line 4:** We cannot pass HTML `<video>` elements to a Web Worker. We must extract the pixels into a generic `ArrayBuffer` first.
- **Line 13 (TinyFaceDetector):** We utilize the lightweight quantized model to ensure inference remains under 100ms on edge hardware, prioritizing speed over extreme accuracy, which is balanced out by the initial QR code identity verification.